

CGaLore: Curvature-Guided GaLore for Memory-Efficient Continual Adaptation of ASR Foundation Models

Steven Vander Eeckt

Department of Electrical Engineering ESAT/PSI
KU Leuven
Leuven, Belgium
steven.vandereeckt@esat.kuleuven.be

Hugo Van hamme

Department of Electrical Engineering ESAT/PSI
KU Leuven
Leuven, Belgium
hugo.vanhamme@esat.kuleuven.be

Abstract—Automatic speech recognition models suffer from catastrophic forgetting when adapted to new domains, accents, or downstream tasks. This problem becomes increasingly important with the growing use of speech foundation models, where adaptation should be both memory-efficient and safe, preserving the broad capabilities learned during pretraining. Gradient Low-Rank Projection (GaLore) has recently been proposed as a memory-efficient fine-tuning method that keeps model parameters full-rank while reducing the optimizer memory through low-rank gradient projection. However, GaLore does not account for catastrophic forgetting. We propose Curvature-Guided GaLore (CGaLore), which incorporates old-task curvature information when selecting the low-rank projection bases. Specifically, CGaLore filters current-task gradients using Kronecker-factored approximate curvature from previous tasks before computing the gradient subspace. Our experiments show that CGaLore enables effective adaptation while alleviating forgetting, outperforming state-of-the-art continual learning baselines. Extensive ablation studies confirm the practical applicability of CGaLore.

Index Terms—automatic speech recognition, GaLore, continual learning, foundation models

I. INTRODUCTION

Automatic speech recognition (ASR) has increasingly moved toward large foundation models: general-purpose models trained on large and diverse speech corpora, often across many languages, domains, and acoustic conditions. Examples include OWSM [1], Whisper [2], OWSM v4 [3], Granite Speech [4], Qwen3-ASR [5], and recent Canary and Parakeet models [6]. Because of their broad pretraining, these models provide strong starting points for downstream adaptation, making it possible to obtain excellent performance on specific domains, accents, speakers, or tasks.

Adapting such models, however, raises two important challenges. First, foundation models contain many parameters, making full fine-tuning computationally intensive. This has motivated parameter-efficient fine-tuning methods, such as Low-Rank Adaptation (LoRA) [7]. Second, adaptation can lead to catastrophic forgetting [8]: while learning a new task, the model may overwrite previously acquired knowledge. This is undesirable for foundation models, where adaptation should add new expertise without sacrificing the broad capabilities obtained during pretraining. The objective is therefore continual learning (CL): adapting to new tasks while preserving performance on previously learned ones.

The combination of efficiency and CL gives rise to parameter-efficient continual learning (PECL) [9], where the goal is to adapt large pretrained models efficiently and without forgetting. In natural language processing (NLP) and computer vision, PECL has received increasing attention, with several methods building on LoRA and related low-rank adaptation techniques [9]–[13]. In ASR, however, PECL remains comparatively underexplored. [14] combine orthogonal LoRA [15] with AdaLoRA [16], while [17] apply weight averaging [18] to task-specific LoRA modules. These approaches do

not directly address forgetting with respect to the initial foundation model; while task-specific adapter methods [19], [20] typically require the task identity to be known at inference time. Finally, recent Singular Value Decomposition (SVD)-based PECL methods for ASR constrain adaptation to selected singular subspaces [21], [22].

More recently, GaLore [23] has offered an alternative to LoRA-based fine-tuning. Instead of freezing the pretrained weights and learning additional low-rank modules, GaLore keeps the model parameters full-rank but projects full-rank gradients into a much lower-dimensional subspace before applying the optimizer. The optimizer update is then projected back to the original parameter space. Since optimizers often store parameter-shaped states [24]–[26], this substantially reduces optimizer-state memory while still allowing full-rank parameter updates over time. Several recent works have extended GaLore. Q-GaLore [27] further reduces memory through quantization and layer-adaptive subspace updates, while GaLore 2 [28] accelerates basis updates using randomized SVD [29]. Its memory-efficient properties make GaLore a promising basis for memory-efficient continual learning. Recent work in NLP has explored this direction by using optimizer first-order moments [30] or historical gradients [31] to reduce interference with old tasks. However, they require information collected during the initial foundation model training process, typically unavailable when adapting an already released ASR foundation model to a downstream task.

In this work, we propose Curvature-Guided GaLore (CGaLore), a memory-efficient CL method for safe adaptation of foundation models. CGaLore uses Kronecker-factored approximate curvature (KFAC) estimated on old-task data with the initial foundation model to approximate the old-task Hessian. This curvature information is used to filter the current-task gradient before computing the projection bases. As a result, the low-rank optimizer subspace is guided toward directions that are relevant for the new task while being less harmful to previous tasks. CGaLore therefore preserves the main advantage of GaLore—memory-efficient full-rank adaptation—while explicitly biasing the adaptation toward directions that reduce forgetting.

Our contributions are as follows. First, we study memory-efficient CL for ASR foundation-model adaptation in the practical setting where a pretrained model is available, but pretraining gradients, optimizer states, and full pretraining data are not. Second, we propose CGaLore, a curvature-guided extension of GaLore that uses old-task KFAC to guide the low-rank optimizer subspaces during training. Third, we show that CGaLore improves the learning–retention trade-off over state-of-the-art baselines, substantially reducing GaLore’s forgetting while preserving its efficiency. Finally, we analyze the role of KFAC estimation data, projection structure, and post-hoc inverse-Hessian correction, showing that CGaLore can work with limited old-task data and can also reduce forgetting on held-out old tasks.

II. PROBLEM FORMULATION & BACKGROUND

A. ASR Model & Notation

We consider an ASR (foundation) model with parameters $\theta \in \mathbb{R}^N$ that maps a speech utterance $\mathbf{X} \in \mathbb{R}^{F \times d_s}$ to a target sequence $\mathbf{y}$ of w output tokens. F denotes the number of acoustic frames and d_s the feature dimension per frame. Given a paired training example $(\mathbf{X}, \mathbf{y})$, the model is optimized by minimizing the ASR loss $\mathcal{L}(\mathbf{X}, \mathbf{y}; \theta)$. The adaptation methods considered in this work operate on linear weight matrices of the model. We denote such a matrix by $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, with d_{in} the input and d_{out} the output dimension.

B. Continual Learning

We assume the initial model, with parameters θ^0 , has been trained on an initial task set $\mathcal{T}_0$. The model is adapted sequentially to a stream of new tasks $\mathcal{T}_1, \dots, \mathcal{T}_t$. At each stage $i > 0$, adaptation starts from the parameters θ^{i-1} obtained after the previous task and produces updated parameters θ^i . The objective is to perform well on the current task $\mathcal{T}_i$, but also to preserve performance on all previously seen tasks $\mathcal{T}_0, \dots, \mathcal{T}_{i-1}$. Each task $\mathcal{T}_i$ is associated with a dataset $(\mathbf{X}, \mathbf{y}) \in \mathcal{D}_i$ of paired speech. In the CL setting studied here, data from previous tasks, $\mathcal{D}_0, \dots, \mathcal{D}_{i-1}$, is no longer available when adapting to $\mathcal{T}_i$, or is too costly to reintroduce at scale. This is particularly relevant for foundation models, whose initial training involves hundreds of thousands of hours of speech [3]. The central challenge is therefore to acquire new task-specific knowledge while avoiding catastrophic forgetting of the capabilities present in the initial model.

C. Parameter Efficient Fine-Tuning

In addition to avoiding forgetting, adaptation should be memory- and compute-efficient. Since foundation models are generally large, fully fine-tuning all parameters is costly. Parameter-efficient methods aim to reduce this overhead by adapting only a small number of additional parameters. Most parameter-efficient fine-tuning methods focus on the linear weight matrices $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ of the model and adapt them while keeping the remaining weights frozen.

Low-Rank Adaptation (LoRA) [7] trains a low-rank update $\Delta \mathbf{W} = \mathbf{B}\mathbf{A}$, with $\mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}}$, where $r \ll \min(d_{\text{out}}, d_{\text{in}})$, and adds this update to $\mathbf{W}$. Another example is Structured SVD (SSVD) [21], which considers the SVD of $\mathbf{W}$ and introduces rescaling and rotation matrices operating on the top- k singular directions, while freezing the singular values and vectors.

D. Gradient Low-Rank Projection with GaLore

Gradient Low-Rank Projection (GaLore) [23] builds on the observation that, although a weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ need not be low-rank, its gradients often exhibit low-rank structure. GaLore exploits this property by applying the optimizer in a low-dimensional gradient subspace, thereby reducing the memory required by optimizer states such as those used by Adam [24], AdamW [25], and Adagrad [26], whose states typically scale with the number of parameters and can exceed the memory needed to store the parameters themselves [23].

Let $\mathbf{G}_s \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ denote the gradient of weight matrix $\mathbf{W}$ at optimization step s :

$$\mathbf{G}_s = -\frac{\partial \mathcal{L}(\mathbf{X}_s, \mathbf{y}_s; \theta_{s-1})}{\partial \mathbf{W}}. \quad (1)$$

where $(\mathbf{X}_s, \mathbf{y}_s)$ is the paired sample or mini-batch at step s and θ_{s-1} are the model parameters from step $s-1$.

Given orthogonal matrices $\mathbf{P} \in \mathbb{R}^{d_{\text{out}} \times r}$ and $\mathbf{R} \in \mathbb{R}^{d_{\text{in}} \times r}$, with $r \ll \min(d_{\text{out}}, d_{\text{in}})$, GaLore projects the gradient into an $r \times r$ optimizer space:

$$\tilde{\mathbf{G}}_s = \mathbf{P}^\top \mathbf{G}_s \mathbf{R} \in \mathbb{R}^{r \times r} \quad (2)$$

The optimizer is applied to $\tilde{\mathbf{G}}_s$, i.e., $\tilde{\mathbf{G}}_s^{\text{OPT}} = \text{Optimizer}(\tilde{\mathbf{G}}_s)$, so that its states are stored in the $r \times r$ projected space rather than in the original $d_{\text{out}} \times d_{\text{in}}$ space. The resulting optimizer update $\tilde{\mathbf{G}}_s^{\text{OPT}}$ is then mapped back to the original parameter space:

$$\mathbf{G}_s^{\text{OPT}} = \mathbf{P} \tilde{\mathbf{G}}_s^{\text{OPT}} \mathbf{R}^\top, \quad (3)$$

and used to update the weight matrix $\mathbf{W}_{s-1}$ from θ_{s-1} to $\mathbf{W}_s$:

$$\mathbf{W}_s = \mathbf{W}_{s-1} + \eta \mathbf{G}_s^{\text{OPT}}, \quad (4)$$

with η the learning rate. Thus, unlike PEFT methods, GaLore does not reduce the number of trainable parameters, nor is its adaptation low-rank; instead, it achieves memory-efficient full-parameter adaptation by reducing the dimensionality of the optimizer states.

To construct the projection matrices, GaLore computes the singular value decomposition of the current gradient,

$$\mathbf{G}_s = \mathbf{U}_s \Sigma_s \mathbf{V}_s^\top = \sum_{j=1}^k \sigma_{s,j} \mathbf{u}_{s,j} \mathbf{v}_{s,j}^\top, \quad (5)$$

where $k = \min(d_{\text{out}}, d_{\text{in}})$. The projection matrices are chosen as the leading singular vectors,

$$\mathbf{P} = \mathbf{U}_{s,:r}, \quad \mathbf{R} = \mathbf{V}_{s,:r}. \quad (6)$$

Since recomputing the SVD at every step is costly (and suboptimal [23]), GaLore keeps $\mathbf{P}$ and $\mathbf{R}$ fixed for T optimization steps and updates them using the SVD of gradient $\mathbf{G}_{s+T}$ after this interval.

III. CURVATURE-GUIDED GALORE

GaLore reduces optimizer-state memory, but its projection bases are selected from the current gradient alone and provide no protection against forgetting. We propose Curvature-Guided GaLore (CGaLore), which uses old-task curvature to guide the construction of the low-rank optimizer subspace. CGaLore is summarized in Algorithm 1.

A. Integrating Old-Task Curvature Information

Let S denote some data from the initial task set $\mathcal{T}_0$, i.e., $S \subseteq \mathcal{D}_0$, and define the old-task loss as $\mathcal{L}_0(\theta) = \frac{1}{|S|} \sum_{(\mathbf{X}, \mathbf{y}) \in S} \mathcal{L}(\mathbf{X}, \mathbf{y}; \theta)$. Since θ^0 was optimized on $\mathcal{D}_0$, we assume that $\nabla_{\theta} \mathcal{L}_0(\theta^0) \approx \mathbf{0}$. A second-order Taylor expansion around θ^0 then gives

$$\mathcal{L}_0(\theta^0 + \Delta\theta) \approx \mathcal{L}_0(\theta^0) + \frac{1}{2} \Delta\theta^\top \mathcal{B}_0 \Delta\theta, \quad (7)$$

where $\mathcal{B}_0 = \nabla_{\theta}^2 \mathcal{L}_0(\theta^0) \in \mathbb{R}^{N \times N}$ is the Hessian of the old-task loss at the initial model.

Let $\mathbf{g} = -\nabla_{\theta} \mathcal{L}(\mathbf{X}, \mathbf{y}; \theta^0) \in \mathbb{R}^N$ be the descent direction for a sample $(\mathbf{X}, \mathbf{y}) \in \mathcal{D}_1$ from a new task $\mathcal{T}_1$. Updating the model directly along $\mathbf{g}$ may cause forgetting on $\mathcal{T}_0$. We therefore consider the direction that maximizes alignment with $\mathbf{g}$ while keeping the quadratic increase in old-task loss bounded:

$$\max_{\mathbf{p}} \mathbf{p}^\top \mathbf{g} \quad \text{s.t.} \quad \frac{1}{2} \mathbf{p}^\top \mathcal{B}_0 \mathbf{p} \leq \tau. \quad (8)$$

The solution is proportional to the inverse-Hessian direction, i.e. $\mathbf{p} \propto \mathcal{B}_0^{-1} \mathbf{g}$: multiplying the new-task gradient by the inverse old-task Hessian yields a direction aligned with the new task that limits the quadratic increase in old-task loss. As CGaLore uses this result only to identify directions that are useful for the new task and safe for the old tasks, the proportionality constant τ is irrelevant.

In practice, forming the Hessian $\mathcal{B}_0$ is infeasible. Prior work approximates the Hessian with Kronecker-factored approximate curvature (KFAC) [32] and applies the inverse-Hessian correction at layer level [33]. Importantly, [33] applies the correction post hoc to the final parameter displacement after fine-tuning. In contrast, we use KFAC during optimization to guide the projection bases.

B. Curvature-Guided Subspace Selection

Consider $\mathbf{W}^0$ the parameters of the weight matrix $\mathbf{W}$ from the initial model parameters θ^0 . The Hessian $\mathbf{B}$ of the loss on old tasks $\mathcal{T}_0$ around $\mathbf{W}^0$, approximated using KFAC, is given by:

$$\mathbf{B} = \frac{\partial^2 \mathcal{L}_0(\theta)}{\partial(\mathbf{W}^0)^2} \approx \mathbf{Q} \otimes \mathbf{H} \quad (9)$$

with $\mathbf{Q} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{in}}}$ the input covariance and $\mathbf{H} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{out}}}$ output-gradient covariance, so that $\mathbf{Q} \otimes \mathbf{H}$ is positive-definite.

To update the orthogonal bases $\mathbf{P}$ and $\mathbf{R}$ every T steps, CGaLore starts from the current gradient $\mathbf{G}_s$ and applies the inverse old-task curvature estimate from Eq. (9), following Sec. III-A, before computing the SVD. At the layer level, using the KFAC, this gives

$$\mathbf{G}_s^{\text{KFAC}} = \mathbf{H}^{-1} \mathbf{G}_s \mathbf{Q}^{-1} \quad (10)$$

Next, as in Eq. (5), we apply SVD to the resulting gradient:

$$\mathbf{G}_s^{\text{KFAC}} = \mathbf{U}_s^{\text{KFAC}} \Sigma_s^{\text{KFAC}} (\mathbf{V}_s^{\text{KFAC}})^\top \quad (11)$$

and, following Eq. (6), find $\mathbf{P}$ and $\mathbf{R}$ by keeping only the r singular directions associated with highest singular values. The resulting projection matrices $\mathbf{P}$ and $\mathbf{R}$ therefore span directions that are both relevant for the new task, because they are derived from the current-task gradient, and less likely to interfere with old tasks, because the gradient is filtered by the inverse old-task curvature before the singular directions are selected.

For any gradient $\mathbf{G}_p$ with $p > s$, we now proceed as in Eqs. (2)–(4) from GaLore. As in GaLore, moreover, we update $\mathbf{P}, \mathbf{R}$ every T iterations using Eqs. (10), (11) and (6).

C. Two-Sided Projection

While GaLore can project only on the left or right side of $\mathbf{G}_s$ in Eq. (2), CGaLore uses the full projection. This allows the update subspace to be restricted in both the output and input dimensions of $\mathbf{W}$. Since CGaLore uses these bases not only for memory reduction but also to bias updates toward directions with low old-task curvature, constraining both sides gives a more complete layer-wise control over the update than using only $\mathbf{P}$ or only $\mathbf{R}$.

D. Maintaining KFAC statistics

Foundation models are often trained on hundreds of thousands of hours of speech [3], making it impractical to compute KFAC statistics on all pretraining data. For the initial adaptation, as described in Sec. III-A, we therefore sample a subset $S \subseteq \mathcal{D}_0$ and compute the KFAC factors $\mathbf{Q}$ and $\mathbf{H}$ on the initial model θ^0 , yielding the layer-wise approximation $\mathbf{B} \approx \mathbf{Q} \otimes \mathbf{H}$ for each weight matrix $\mathbf{W}^0$.

After adapting to a subsequent task $\mathcal{T}_i$, we sample a subset $S_i \subseteq \mathcal{D}_i$ and compute task-specific KFAC factors $\tilde{\mathbf{Q}}_i$ and $\tilde{\mathbf{H}}_i$ using the adapted model. We then update the stored KFAC factors by averaging:

$$\mathbf{Q} \leftarrow (1 - \alpha)\mathbf{Q} + \alpha\tilde{\mathbf{Q}}_i, \quad \mathbf{H} \leftarrow (1 - \alpha)\mathbf{H} + \alpha\tilde{\mathbf{H}}_i, \quad (12)$$

with $0 \leq \alpha \leq 1$. This provides a compact approximation of the accumulated old-task curvature: instead of storing separate KFAC factors for every previous task, CGaLore keeps only one running set of factors per layer, which can later be used to adapt to task $\mathcal{T}_{i+1}$.

E. Implementation and Efficiency

CGaLore uses KFAC only when updating the GaLore projection bases. The KFAC factors are precomputed from old-task data and stored in CPU memory; at an update step, the factors for the current layer are temporarily moved to the GPU to compute Eq. (10). Thus, KFAC does not increase the GPU memory footprint, but only adds

computation during the subspace update. To reduce the update cost, we use randomized SVD [29] instead of exact SVD, following [28].

Algorithm 1 Curvature-Guided GaLore (CGaLore)

Require: Previous model θ^{i-1} , data $\mathcal{D}_i$ for new task $\mathcal{T}_i$, KFAC factors $\mathbf{Q}, \mathbf{H}$, rank r , update interval T

- 1: Set $s \leftarrow 1$, $\theta_{s-1} \leftarrow \theta^{i-1}$
- 2: **for** each optimization step s on $(\mathbf{X}, \mathbf{y}) \in \mathcal{D}_i$ **do**
- 3: **for** each linear layer with weights $\mathbf{W}_{s-1} \in \theta_{s-1}$ **do**
- 4: Compute gradients $\mathbf{G}_s$ using Eq. (1).
- 5: **if** $s = 1$ or $s \bmod T = 1$ **then**
- 6: Update $\mathbf{P}, \mathbf{R}$ through Eqs. (10), (11) and (6).
- 7: **end if**
- 8: Update $\mathbf{W}_{s-1}$ into $\mathbf{W}_s$ using Eqs. (2), (3) and (4).
- 9: Store $\mathbf{W}_s$ in θ_s
- 10: **end for**
- 11: **end for**
- 12: Update KFAC factors $\mathbf{Q}, \mathbf{H}$ using data $S_i \subseteq \mathcal{D}_i$ and Eq. (12).
- 13: **return** Adapted model parameters $\theta^i = \theta_s$

IV. EXPERIMENTS

Experiments are done in ESPnet2 [34]. Code, configurations, and data splits are available at github.com/StevenVdEeck/cgalore.

Model. The model is the Open Whisper-style Speech Model (OWSM) v3.2 small [1], consisting of 9 E-Branchformer [35] encoder and 9 Transformer [36] decoder layers. The model, with a vocabulary of 50,000 output tokens, containing 366.7M parameters, was trained on 180k hours of speech from 151 languages. For adaptation, only the weight matrices $\mathbf{W}$ of the linear layers are updated, while output layers are frozen. We integrate (C)GaLore with AdamW [25] (using learning rate $5e-4$) without scheduler, training the model for 20 epochs with an effective batch size of 64.

Data. We conduct two experiments. For both, the initial tasks $\mathcal{T}_0$ are English (ENG), German (DEU), and Spanish (ESP) from Common Voice [37], used to measure forgetting. In Exp. 1, we adapt OWSM to L2-Arctic [38], which contains English speech from non-English speakers. We split L2-Arctic into two tasks based on accent, referred to as L2/1 and L2/2. The second experiment resembles [39], using Corpus Gesproken Nederlands (CGN) [40], split into Dutch from the Netherlands (CNL) and Belgium (CVL), yielding two tasks. CGN includes varied Dutch speech from settings such as interviews, lectures, and broadcasts.

Baselines. We compare against the following methods and describe how each adapts linear layer $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ to the new task:

- (a) LoRA [7]: learns a low-rank adaptation as discussed in Sec. II-C, without tackling catastrophic forgetting.
- (b) LoRA + FTA [17], [22]: combines LoRA with Fine-Tuning with Averaging (FTA) [18], averaging the low-rank updates $\Delta \mathbf{W}$ with the pretrained weight matrix $\mathbf{W}$.
- (c) GaLore [23], discussed in Sec. II-D.
- (d) BiLoRA [10]: uses a bilinear update in fixed orthogonal bases, i.e., $\Delta \mathbf{W} = \mathbf{F}_{\text{out}} \mathbf{B} \mathbf{F}_{\text{in}}^H$, with $\mathbf{F}_{\text{out}}$ and $\mathbf{F}_{\text{in}}$ the 1D discrete Fourier transform matrices so that only k_B elements in $\mathbf{B} \in \mathbb{C}^{d_{\text{out}} \times d_{\text{in}}}$ are trained.
- (e) Continual SSVD (CSSVD) [22]: builds on SSVD [21], explained in Sec. II-C, instead learning only rotations between the bottom- k singular directions on the new task.
- (f) Full Fine-Tuning (FFT): we also fully fine-tune the linear weight matrices, which will result in forgetting. As additional baselines,

TABLE I

RESULTS OF THE EXPERIMENTS. “METHOD” DENOTES THE BASE ADAPTATION METHOD, WHILE “CL” DENOTES THE CONTINUAL LEARNING METHOD APPLIED ON TOP OF IT. WERS ARE MEASURED AFTER LEARNING ALL TASKS. NEGATIVE BWT INDICATES FORGETTING.

		Experiment 1								Experiment 2							
Method	CL	WER↓ per task					Average		WER↓ per task					Average			
		1-ENG	1-DEU	1-ESP	2-L2/1	3-L2/2	WER↓	BWT↑	1-ENG	1-DEU	1-ESP	2-CNL	3-CVL	WER↓	BWT↑		
Initial model	–	13.4	11.3	11.3	8.2	16.7	12.18	–	13.4	11.3	11.3	45.7	37.9	22.48	–		
Full Fine-Tuning	–	21.0	17.1	15.9	1.7	2.3	11.60	-4.4	24.3	57.3	21.7	27.7	13.6	28.94	-18.2		
	IHR	13.4	11.2	11.3	3.3	7.9	9.39	0.3	13.7	11.6	11.6	31.8	23.4	18.41	1.0		
	SVR	15.2	12.2	11.8	1.6	3.6	8.88	-0.5	16.9	22.7	15.0	27.5	15.6	19.52	-4.6		
	Sep. Model	13.4	11.3	11.3	2.1	2.3	8.07	0.0	13.4	11.3	11.3	22.4	13.6	14.38	0.0		
LoRA	–	25.1	31.2	27.9	2.2	2.6	17.79	-11.9	41.8	88.4	44.5	28.9	14.7	43.66	-35.7		
	FTA	14.4	12.8	12.4	3.0	7.5	10.01	-0.6	16.4	20.6	15.1	27.2	19.0	19.64	-3.6		
GaLore	–	20.8	17.6	16.9	1.8	2.7	11.98	-4.7	23.7	53.1	22.2	30.6	15.1	28.93	-17.2		
BiLoRA	–	18.1	14.7	14.3	1.7	3.1	10.37	-2.5	19.4	34.7	17.8	30.0	16.0	23.58	-9.7		
CSSVD	–	14.1	11.9	12.0	2.8	6.8	9.51	-0.3	14.9	14.4	13.4	28.6	20.4	18.33	-1.9		
CGaLore	–	15.1	11.5	11.6	2.0	3.5	8.72^a	-0.3	15.5	14.5	13.0	28.9	16.4	17.68^a	-2.7		

^a Significantly outperforms all baselines except the full fine-tuning separate-model reference, which serves as an upper bound.

we combine FFT with (a) Inverse Hessian Regularization (IHR) [33], which applies the inverse curvature correction post-hoc to the update ΔW ; (b) Singular Value Rehearsal [39], which applies SVD to the update ΔW and learns on a small memory buffer of old data which directions to keep; (c) Sep. Model, which provides a best-case scenario, keeping a separate model θ^i per task $\mathcal{T}_i$ using a task oracle - it does therefore not forget. SVR is a rehearsal-based method, requiring storage of past data.

PECL methods learn 9.0M, and FFT and (C)GaLore 244.8M parameters; with $r = 20$ for LoRA-based methods and $p = 0.40$ for CSSVD. For SVR, as in [39], we store 50 utterances (approx. 4 minutes) in the memory buffer, divided equally across old tasks.

Implementation. For (C)GaLore, we set $r = 215$, the optimizer states thus having approximately the same memory budget as PECL baselines, and update the projection bases every $T = 200$ optimization steps. For CGaLore, KFAC factors are estimated using all three old tasks, with 20k utterances per task, corresponding to 28 hours of speech per task, and $\alpha = 1/4$. We use KFAC damping of 10^{-3} .

Metrics. We report word error rate (WER, %) for the final model on each task, and use Average WER over all learned tasks as main metric. Forgetting is measured by backward transfer (BWT), defined as: $BWT = 1/i \sum_{j=0}^{i-1} (WER_j^i - WER_j^j)$, where WER_k^l represents WER on task $\mathcal{T}_k$ after learning task $\mathcal{T}_l$, and $\mathcal{T}_i$ is the most recently learned task. In our setting, $\mathcal{T}_0$ contains multiple tasks (Sec. II-A), which are treated individually when computing BWT. Negative values indicate mean WER increase and degradation on old tasks. Average WER significance is assessed with a Wilcoxon signed-rank test on per-utterance error counts [41] at 0.1% level.

V. RESULTS

Table I shows the results of the two experiments.

A. Experiment 1: L2-Arctic

Exp. 1 finds all methods substantially improving performance on L2-Arctic compared with the initial model. LoRA and GaLore reach new-task WERs close to full fine-tuning, but differ strongly in forgetting: LoRA suffers much larger degradation on the initial tasks, whereas GaLore reduces this effect but still obtains a BWT of -4.7.

CGaLore gives the best balance between learning and retention. It achieves the lowest forgetting among parameter- and memory-efficient methods, with BWT of -0.3, while also learning the new tasks better than LoRA+FTA and CSSVD. As a result, CGaLore obtains the best Average WER of 8.72, improving over CSSVD, the strongest baseline, by 8.3%. Compared with GaLore, CGaLore eliminates 93.6% of forgetting while recovering 94.3% of the L2/2 performance gap, closing 83.4% of the gap between GaLore and the Full Fine-Tuning + Sep. Model reference. CGaLore even outperforms all FFT baselines, including SVR which relies on storage of past data.

B. Experiment 2: CGN

Similar conclusions hold for Exp. 2, although forgetting is on average substantially higher for most methods. LoRA and GaLore again approach the new-task performance of full fine-tuning, but both suffer strong degradation on the initial tasks.

CGaLore again achieves the best average WER, with 17.68. It provides the strongest balance between learning and retention: compared with CSSVD, it has slightly higher forgetting but better new-task performance. Compared with GaLore, CGaLore reduces forgetting by 84.3% while retaining 94.3% of the CVL performance, and closes 77.4% of the gap between GaLore and the FFT separate-model reference. CGaLore also outperforms all FFT baselines

C. Effect of KFAC Estimation Data

Table II reports WERR (WER reduction) on old tasks after the first adaptation of Exp. 1 and Exp. 2. Rows indicate which old-task data are used to compute KFAC. We report WERR both for the main old tasks used in the primary evaluation (ENG, DEU, ESP) and for held-out old tasks not used for KFAC: TEDLIUM [42] (TED), Swedish (SWE), Italian (ITA), and Russian (RUS) from Common Voice. Negative WERR indicates forgetting.

Moving from GaLore, which uses no KFAC, to CGaLore substantially reduces forgetting on both main and held-out old tasks. In Exp. 1, using ENG for KFAC further reduces forgetting on ENG, while the effect of including DEU is small. For ESP, using only ENG appears to bias the update toward directions safe for ENG and DEU but less safe for ESP. The held-out tasks generally follow their closest main-task counterpart: ITA follows ESP, SWE behaves similarly to DEU, and RUS benefits most from the more diverse KFAC estimate.

TABLE II

WERR (WER REDUCTION, %) ON OLD TASKS AFTER ADAPTATION. ROWS INDICATE THE OLD-TASK DATA USED TO COMPUTE KFAC; “-” DENOTES GALORE WITHOUT KFAC. NEGATIVE WERR INDICATES FORGETTING. WERR IS GIVEN FOR MAIN (ENG, DEU, ESP) AND HELD-OUT OLD TASKS NOT USED FOR KFAC: TEDLIUM (TED) [42], SWEDISH (SWE), ITALIAN (ITA) AND RUSSIAN (RUS) FROM COMMON VOICE.

Tasks for KFAC	Per main task			Per held-out task			
	ENG	DEU	ESP	TED	SWE	ITA	RUS
<i>Experiment 1 – Adaptation to 2–L2/1</i>							
–	-18.7	-16.8	-15.0	-7.0	-15.0	-17.7	-13.7
ENG+DEU+ESP	-3.7	-0.9	-2.7	-4.1	-3.2	-3.2	-1.0
ENG	-0.7	-0.9	-6.2	-2.2	-4.5	-5.5	-4.0
DEU	-6.7	-1.8	-2.7	-4.5	-3.9	-2.9	-3.5
<i>Experiment 2 – Adaptation to 2–CNL</i>							
–	-38.1	-134.5	-46.0	-34.0	-51.7	-56.3	-60.5
ENG+DEU+ESP	-10.4	-15.9	-6.6	-14.4	-23.5	-9.8	-6.7
ENG	-9.7	-40.7	-13.3	-12.0	-28.9	-15.7	-12.3
DEU	-14.2	-9.7	-14.2	-16.2	-22.6	-11.0	-6.0

In Exp. 2, including DEU in the KFAC estimate is especially important. Since the new Dutch task is close to German, interference with DEU is high, as also reflected by the strong forgetting of FFT, LoRA, and GaLore in Table I. Using DEU for KFAC therefore greatly reduces forgetting on DEU. SWE, another related language, follows the same trend, although the reduction is less pronounced; using SWE additionally for KFAC might further reduce forgetting.

These results show that CGaLore also reduces forgetting on held-out old tasks that were not used for KFAC estimation. This is important for foundation-model adaptation: CGaLore does not require access to all tasks used during pretraining to provide protection against forgetting. Nevertheless, KFAC is most effective when it includes old tasks that strongly interfere with the new task.

Fig. 1 further studies how much data is needed to estimate KFAC. We vary the amount of speech per main old task from 0 seconds, corresponding to GaLore without KFAC, to 28 hours, using ENG, DEU, and ESP for KFAC estimation. We report average WER, BWT on the main old tasks, and BWT on the held-out old tasks.

Reducing the KFAC data from 28 hours to 8.3 minutes per task has little effect: average WER and BWT remain nearly unchanged. Performance starts to degrade only when reducing the data to 50 seconds per task, and forgetting increases further at 5 seconds per task. Even then, CGaLore still reduces GaLore’s forgetting by more than 60%. Thus, while more KFAC data is beneficial, a small amount of old-task speech is sufficient for CGaLore to reduce forgetting.

D. Memory and Runtime Efficiency

Table III studies the training efficiency of CGaLore on the first adaptation of Exp. 1. GaLore and CGaLore do not increase peak GPU memory relative to LoRA: both use 10.56 GB, slightly below LoRA at 10.90 GB, despite updating full weight matrices rather than only LoRA parameters. Although GaLore makes the optimizer step substantially more expensive, with optimizer time $12.6\times$ higher than LoRA, its unchanged forward/backward graph makes total training approximately 11% faster overall in our setting with gradient accumulation of 16. CGaLore adds only modest overhead over GaLore: KFAC multiplication accounts for 19% of the randomized-SVD time, increasing optimizer time by 1.3% and total training time by only 0.4%. Thus, CGaLore retains GaLore’s practical efficiency while adding curvature-guided subspace selection.

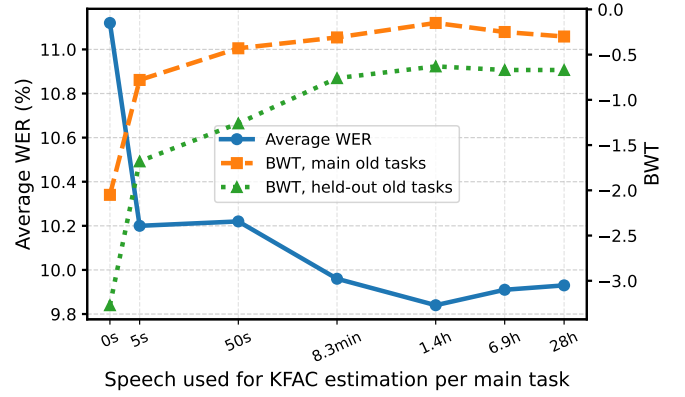


Fig. 1. Performance of CGaLore as a function of speech used (per task) for KFAC estimation the first adaptation of Exp. 1. ENG, DEU and ESP are used for KFAC estimation. The blue curve shows the Average WER (on ENG, DEU, ESP and L2/1), orange the BWT on ENG, DEU and ESP; and green the BWT on TED, SWE, ITA and RUS. Negative BWT indicates forgetting.

TABLE III

TRAINING EFFICIENCY COMPARISON ON FIRST ADAPTATION OF EXP. 1, ON A SINGLE NVIDIA A100 GPU. PEAK GPU MEMORY DENOTES PEAK ALLOCATED MEMORY. RELATIVE TIME IS NORMALIZED TO LoRA.

Method	Peak GPU	Optimizer time	Train time
	GB ↓	relative ↓	relative ↓
LoRA	10.90	1.0×	1.00×
GaLore	10.56	12.6×	0.89×
CGaLore	10.56	12.8×	0.90×

E. Effect of Subspace Update Interval

Fig. 2 studies the effect of the projection update interval T on average WER and BWT for the first adaptation of Exp. 2. Performance is best at $T = 200$, decreases only slightly up to $T = 2000$, and degrades more noticeably for larger intervals. This degradation mainly affects new-task performance, since BWT remains relatively flat across T . Thus, CGaLore is not highly sensitive to the exact update interval, consistent with observations for GaLore [23]; even at $T = 20000$, the gap to CSSVD remains substantial.

F. Interaction with IHR

CGaLore is related to IHR [33], but uses curvature at a different stage. IHR applies a post-hoc correction to the final parameter update $\Delta\mathbf{W}$ after adaptation, i.e., the inverse Hessian correction from Eq. (10) is applied to $\Delta\mathbf{W}$ learned on the new task which is then added

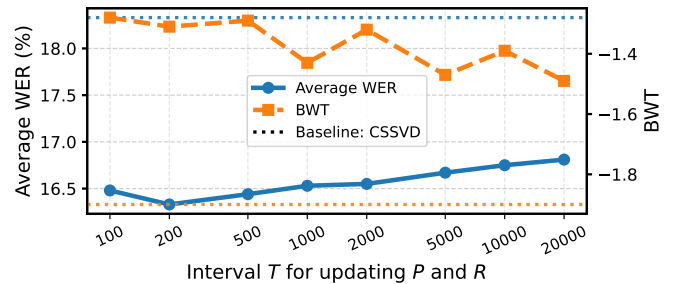


Fig. 2. Average WER and BWT as a function of the interval T in CGaLore on first adaptation of Exp. 2. T determines how often $\mathbf{P}$ and $\mathbf{R}$ are updated.

TABLE IV

COMPARING GaLORE AND CGaLORE WITH AND WITHOUT POST-HOC IHR CORRECTION ON THE FIRST (2–CNL) AND SECOND (3–CVL) ADAPTATION OF EXP. 2. λ IS THE HYPER-PARAMETER CONTROLLING THE ROTATION OF THE UPDATE, AS DISCUSSED IN SEC. V-F.

Method	IHR	λ	2–CNL: Average		3–CVL: Average	
			WER↓	BWT↑	WER↓	BWT↑
GaLore	✗	–	21.56	-8.5	28.93	-17.2
CGaLore	✗	–	16.33 ^a	-1.3	17.68	-2.7
GaLore	✓	0.50	16.58 ^a	-1.2	17.74 ^a	-2.3
CGaLore	✓	0.75	16.09 ^a	-0.6	16.86 ^a	-1.1

^a Difference in Average WER is significant.

TABLE V

EFFECT OF PROJECTION STRUCTURE IN CGaLORE ON THE FIRST ADAPTATION OF EXP. 1. ENG IS USED FOR KFAC ESTIMATION.

Variant	Average	
	WER↓	BWT↑
Full two-sided, $r = 215$	10.00 ^a	-0.3
Left only, $r = 38$	11.40 ^a	-2.2
Right only, $r = 38$	9.92	-0.2

^a Difference in Average WER is significant.

to $\mathbf{W}$; whereas CGaLore uses old-task curvature during training to guide $\mathbf{P}$ and $\mathbf{R}$. Table IV shows the performance of CGaLore and GaLore on the first and second adaptation of Exp. 2 when combined with IHR. When applying IHR, we use a hyper-parameter $\lambda \in [0, 1]$ that controls how far the final update is rotated toward the inverse-Hessian-corrected direction: $\lambda = 0$ leaves the original update unchanged, $\lambda = 1$ applies the full IHR correction. We tune λ on the validation sets after the first adaptation. The update to which Eq. (10) is applied is normalized to equal $\Delta\mathbf{W}$ ’s original norm.

Table IV shows that IHR is a strong post-hoc correction for GaLore, substantially reducing the gap to CGaLore. However, IHR does not make CGaLore redundant: in Exp. 2 after the first adaptation, GaLore+IHR reaches 16.58 WER, whereas CGaLore obtains 16.33 and CGaLore+IHR further improves to 16.09. After the second adaptation, the gap between GaLore and CGaLore when both are combined with IHR becomes even larger, with CGaLore + IHR obtaining an average WER of 16.86, further increasing the gap with baselines from Table I. This suggests that curvature-guided subspace selection during training can provide a better learning–retention trade-off than correcting the final update only after training. At the same time, the best results often come from combining CGaLore with a mild IHR correction, indicating that CGaLore and IHR are complementary rather than mutually exclusive.

G. Left, Right, and Two-Sided Projection

Table V compares the two-sided projection with one-sided variants on the first adaptation of Exp. 1, using only ENG for KFAC. To keep the optimizer-state memory approximately matched, we use $r = 215$ for the two-sided projection and $r = 38$ for the one-sided projections. The results suggest that constraining the input-side subspace is most important in this setting: right projection performs similarly to the full two-sided projection, whereas left projection degrades BWT. This indicates that, for the evaluated accent-adaptation tasks, preventing interference in input/activation-side directions is more important than constraining the output side alone.

VI. DISCUSSION

CGaLore requires old-task data only to estimate KFAC factors, and Sec. V-C shows that this can be done with only a few minutes of speech per task. Moreover, the resulting factors also reduce forgetting on held-out old tasks not used for KFAC estimation. After KFAC has been computed, the old data can be discarded; CGaLore stores curvature factors rather than replay examples. This is a key difference with rehearsal-based methods, which store a subset of the past data to replay during adaptation to new tasks: they must keep this data for replay during adaptation and typically require sufficiently diverse stored data as overfitting on the memory remains a risk [43]. This is particularly limiting for foundation models, where $\mathcal{T}_0$ may contain many tasks and highly diverse data.

CGaLore also differs from recent GaLore extensions [30], [31], which rely on information accumulated during the original training process, which is usually unavailable when adapting a released ASR foundation model. In contrast, CGaLore requires only a small amount of old-task data for KFAC estimation to adapt the model. Compared with PECL methods such as CSSVD [22] or BiLoRA [10], CGaLore reduces forgetting through the optimizer trajectory rather than by restricting adaptation to a fixed parameter-efficient update form.

CGaLore leaves several directions for future work. First, unlike recent GaLore extensions [27], [44], CGaLore does not automatically determine when each layer should update its projection bases $\mathbf{P}$ and $\mathbf{R}$. A layer-wise update mechanism that accounts for both current-task gradient drift and old-task KFAC curvature could further reduce the cost of SVD while preserving performance. Second, CGaLore currently uses the same rank r for each layer. A curvature-aware rank allocation strategy could assign larger subspaces to layers that can be adapted safely and smaller subspaces to less relevant or more interference-prone layers, improving the learning–retention trade-off under the same memory budget. Finally, our experiments are limited to a single 367M-parameter ASR model and two-task adaptation sequences. Future work should evaluate CGaLore on larger models and longer task sequences, as well as extend it to foundation-model adaptation in other domains such as NLP or vision.

VII. CONCLUSION

We propose CGaLore, an extension of GaLore for memory-efficient continual adaptation of ASR foundation models. CGaLore uses old-task Kronecker-factored approximate curvature (KFAC) estimates to select the GaLore projection bases after first preconditioning the current-task gradient with old-task KFAC estimates, biasing the optimizer subspace toward directions that remain useful for the new task while reducing interference with previous tasks. Experiments on two CL scenarios show that CGaLore substantially reduces forgetting compared with GaLore and improves the learning–retention trade-off over parameter-efficient CL baselines, while preserving GaLore’s practical GPU memory efficiency. Our analyses further show that CGaLore can work with only a few minutes of speech per task for KFAC estimation, can also reduce forgetting on held-out old tasks not used to compute KFAC, is not highly sensitive to the projection update interval, and can be combined with post-hoc inverse-Hessian correction for additional gains. These results show that old-task curvature can guide low-rank optimizer subspaces during training, enabling speech foundation models to adapt efficiently while preserving previously learned capabilities.

VIII. ACKNOWLEDGMENTS

Research supported by Research Foundation Flanders (FWO) under grant S004923N of the SBO programme.

REFERENCES

- [1] Y. Peng *et al.*, “Owsm v3.1: Better and faster open whisper-style speech models based on e-branchformer,” in *Interspeech*, pp. 352–356, 2024.
- [2] A. Radford *et al.*, “Robust speech recognition via large-scale weak supervision,” 2022.
- [3] Y. Peng, S. Muhammad, Y. Sudo, W. Chen, J. Tian, C.-J. Lin, and S. Watanabe, “OWSM v4: Improving open whisper-style speech models via data scaling and cleaning,” in *Proceedings of the Annual Conference of the International Speech Communication Association (INTER-SPEECH)*, 2025.
- [4] I. G. S. Team, “Granite 4.1 speech,” 2026.
- [5] X. Shi *et al.*, “Qwen3-asr technical report,” *arXiv preprint arXiv:2601.21337*, 2026.
- [6] M. Sekoyan, N. R. Koluguri, N. Tadevosyan, P. Zelasko, T. Bartley, N. Karpov, J. Balam, and B. Ginsburg, “Canary-1b-v2 and parakeet-tdt-0.6b-v3: Efficient and high-performance models for multilingual asr and ast,” 2025.
- [7] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [8] M. McCloskey *et al.*, “Catastrophic interference in connectionist networks: The sequential learning problem,” vol. 24 of *Psychology of Learning and Motivation*, pp. 109–165, 1989.
- [9] W. Zhao, S. Wang, Y. Hu, Y. Zhao, B. Qin, X. Zhang, Q. Yang, D. Xu, and W. Che, “Sapt: A shared attention framework for parameter-efficient continual learning of large language models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11641–11661, 2024.
- [10] H. Zhu, Y. Zhang, J. Dong, and P. Koniusz, “Bilora: Almost-orthogonal parameter spaces for continual learning,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 25613–25622, 2025.
- [11] F. Qiao and M. Mahdavi, “Merge before forget: A single loRA continual learning via continual merging,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [12] M.-L. Luo, Z.-H. Zhou, Y.-L. Zhang, Y. Wan, T. Wei, and M.-L. Zhang, “KeepLoRA: Continual learning with residual gradient adaptation,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [13] Y. Zheng, Y. Zhang, J. van de Weijer, G. M. van de Ven, S. Du, X. Zhang, and Z. Tian, “Revisiting weight regularization for low-rank continual learning,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [14] T. Xu *et al.*, “Towards rehearsal-free multilingual asr: A lora-based case study on whisper,” in *Interspeech 2024*, pp. 2534–2538, 2024.
- [15] X. Wang, T. Chen, Q. Ge, H. Xia, R. Bao, R. Zheng, Q. Zhang, T. Gui, and X. Huang, “Orthogonal subspace learning for language model continual learning,” in *Findings of the Association for Computational Linguistics: EMNLP 2023* (H. Bouamor, J. Pino, and K. Bali, eds.), (Singapore), pp. 10658–10671, Association for Computational Linguistics, Dec. 2023.
- [16] Q. Zhang *et al.*, “Adaptive budget allocation for parameter-efficient fine-tuning,” in *11th International Conference on Learning Representations*, 2023.
- [17] E. Ugan, N.-Q. Pham, and A. Waibel, “Weight Factorization and Centralization for Continual Learning in Speech Recognition,” in *Interspeech 2025*, pp. 2200–2204, 2025.
- [18] S. Vander Eeck and H. Van hamme, “Weight averaging: A simple yet effective method to overcome catastrophic forgetting in automatic speech recognition,” in *ICASSP 2023*, 2023.
- [19] N. Houlsby *et al.*, “Parameter-efficient transfer learning for NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, pp. 2790–2799, 2019.
- [20] S. Vander Eeck and H. Van hamme, “Using adapters to overcome catastrophic forgetting in end-to-end automatic speech recognition,” in *ICASSP 2023*, 2023.
- [21] P. Wang, S. Watanabe, and H. Van hamme, “Ssvd: Structured svd for parameter-efficient fine-tuning and benchmarking under domain shift in asr,” in *Proceedings of the IEEE ASRU Workshop*, 2025.
- [22] S. Vander Eeck and H. Van hamme, “Parameter-efficient continual learning for automatic speech recognition,” in *Proc. Interspeech 2026*, 2026.
- [23] J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar, and Y. Tian, “Galore: Memory-efficient llm training by gradient low-rank projection,” 2024.
- [24] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *3rd International Conference on Learning Representations*, 2015.
- [25] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [26] J. Duchi, E. Hazan, and Y. Singer, “Adaptive subgradient methods for online learning and stochastic optimization,” *Journal of Machine Learning Research*, vol. 12, no. 61, pp. 2121–2159, 2011.
- [27] Z. Zhang, A. Jaiswal, L. Yin, S. Liu, J. Zhao, Y. Tian, and Z. Wang, “Q-galore: Quantized galore with int4 projection and layer-adaptive low-rank gradients,” 2024.
- [28] D. Su, A. Gu, J. Xu, Y. Tian, and J. Zhao, “Galore 2: Large-scale llm pre-training by gradient low-rank projection,” *ArXiv*, vol. abs/2504.20437, 2025.
- [29] N. Halko, P.-G. Martinsson, and J. A. Tropp, “Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.
- [30] C. Wang, Y. Lyu, Z. Sun, and L. Jing, “Continual gradient low-rank projection fine-tuning for LLMs,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, eds.), (Vienna, Austria), pp. 14815–14829, Association for Computational Linguistics, July 2025.
- [31] Q. Cheng, Y. Wan, L. Wu, C. Hou, and L. Zhang, “Continuous subspace optimization for continual learning,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- [32] H. Ritter, A. Botev, and D. Barber, “Online structured laplace approximations for overcoming catastrophic forgetting,” in *Advances in Neural Information Processing Systems* (S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, eds.), vol. 31, Curran Associates, Inc., 2018.
- [33] S. Vander Eeck and H. Van hamme, “Inverse-hessian regularization for continual learning in asr,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026.
- [34] S. Watanabe *et al.*, “ESPnet: End-to-end speech processing toolkit,” in *Proceedings of Interspeech*, pp. 2207–2211, 2018.
- [35] K. Kim *et al.*, “E-branchformer: Branchformer with enhanced merging for speech recognition,” in *2022 IEEE Spoken Language Technology Workshop (SLT)*, pp. 84–91, 2023.
- [36] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [37] R. Ardila *et al.*, “Common voice: A massively-multilingual speech corpus,” in *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC 2020)*, 2020.
- [38] G. Zhao, S. Sonsaat, A. Silpachai, I. Lucic, E. Chukharev-Hudilainen, J. Levis, and R. Gutierrez-Osuna, “L2-arctic: A non-native english speech corpus,” in *Proc. Interspeech*, p. 2783–2787, 2018.
- [39] S. Vander Eeck and H. Van hamme, “Efficient rehearsal for continual learning in asr via singular value tuning,” *IEEE Transactions on Audio, Speech and Language Processing*, vol. 34, pp. 978–991, 2026.
- [40] N. Oostdijk, “The spoken dutch corpus: Overview and first evaluation,” *Proceedings of LREC-2000, Athens*, vol. 2, 01 2000.
- [41] H. Strik, C. Cucchiari, and J. M. Kessens, “Comparing the recognition performance of csrs: in search of an adequate metric and statistical significance test,” in *INTERSPEECH*, 2000.
- [42] F. Hernandez *et al.*, “Ted-lium 3: Twice as much data and corpus repartition for experiments on speaker adaptation,” in *Speech and Computer*, pp. 198–208, 2018.
- [43] E. Verwimp, D. Lange, *et al.*, “Rehearsal revealed: The limits and merits of revisiting samples in continual learning,” in *2021 IEEE/CVF International Conference on Computer Vision*, pp. 9365–9374, 2021.
- [44] T. Miao, Z. Bao, and L. Zhang, “Lotus: Efficient llm training by randomized low-rank gradient projection with adaptive subspace switching,” in *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 4696–4700, 2026.